

PRIMORDIA

A 3D emergence laboratory: two-scale particle life, Barnes–Hut gravity and
a piloted articulated creature

Interactive Graphics

Author name (matricola)

September 6, 2026



Table of Contents

Overview

► Overview

► Simulation engine

► Creature and visualisation

► Interactions

► Wrap-up



The idea

Overview

Complexity does not require complicated rules.

- Thousands of particles, **two force laws**, nothing else
- **Short-range “chemistry”** → cells, membranes, filaments, worms
- **Long-range gravity** → collapse, clumps, orbits, spiral arms
- The user never builds structures, only **edits the rules**

Satisfied constraints

- Everything procedural: no meshes, no images, no animation clips
- Everything live: no parameter change requires a reload
- The acceleration structures are on stage, not hidden



Environment

Overview

- **WebGL** through **three.js**, WebGL 2 when available
- `antialias`, high-performance GPU hint, sRGB output
- **Plain ES6 JavaScript**, no bundler nor transpiler
- Single-threaded, inside the animation-frame callback



What we did *not* write

Overview

One runtime dependency: `three.js r128` (single `<script>` from a CDN)

renderer • scene graph • camera • primitives • `MeshStandardMaterial` • `InstancedMesh` • lights • `CanvasTexture` • `BufferGeometry` • math types

Everything else is ours

- No UI framework, no `dat.GUI`, no physics engine, no `gl-matrix`, no bundler
- Not even `OrbitControls`: our camera shares mouse buttons with the poke tools

Credit: *particle life* / “Clusters” concept (Ventrella) • Barnes–Hut algorithm — implementations ours.

Zero assets

- No `.obj/.glTF/.fbx`, no rigs, no keyframes, no images, no web fonts
- Geometry from parametric primitives, textures painted into a canvas at start-up



Table of Contents

Simulation engine

► Overview

► **Simulation engine**

► Creature and visualisation

► Interactions

► Wrap-up



Architecture

Simulation engine

- 1 HTML + 1 CSS + **10 JS files**
- Load order is the dependency graph:
config → scene → particles → creature → grid → octree → simulation →
camera → ui → main

There is no Particle **object**, everything is an array

- Parallel typed arrays: positions, velocities, type, size, mass
- Cache locality: the inner loop streams, it does not chase pointers
- Allocated once ⇒ **zero allocations per frame** ⇒ the GC never pauses the animation
- Predictable types: JIT emits unboxed floating-point code for the whole loop
- Same discipline for the octree: 13 typed arrays, not a tree of node objects



Two scales, two data structures

Simulation engine

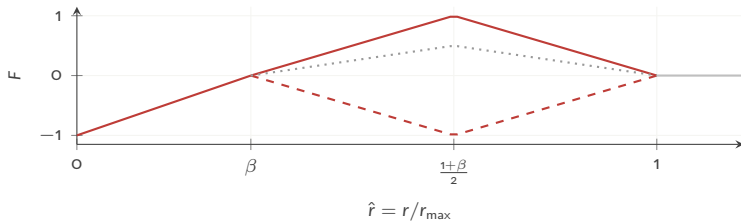
	Chemistry	Gravity
Range	short, hard cut-off	infinite
Depends on	type (matrix)	mass
Naive cost	$O(N^2)$	$O(N^2)$
Structure	uniform grid	Barnes-Hut octree
Why valid	compact support	distant cluster $\approx$ its centre of mass

This pairing is the engineering core of the project.
Target: 1000–3000 mutually interacting particles at 60 Hz.



Chemistry: the force profile

Simulation engine



Force graph is piecewise linear, with three regimes:

1. $\hat{r} < \beta$: type-independent repulsion core
2. $\beta \leq \hat{r} < 1$: type-dependent interaction scaled by the matrix entry (the matrix is asymmetric)
3. $\hat{r} \geq 1$: the interaction is exactly zero beyond $r_{\max}$

Newton's third law is violated on purpose. Red chases green, green flees red: the pair self-propels. That asymmetry is the engine of every moving pattern.



Uniform grid

Simulation engine

- Cell size = interaction radius $\Rightarrow$ search only the **27 neighbouring cells**
- Rebuilt from scratch **every frame** by a **counting sort** for placing particles of the same cell **contiguous in memory**
- Build $O(N + \text{cells})$ allocation-free

At $N = 3000$

$9 \cdot 10^6$ pair tests per frame $\rightarrow$ a few tens of thousands.
The HUD reports the pairs actually evaluated, live.



Gravity: Barnes-Hut octree

Simulation engine

- Pooled arrays indexed by node id, sized once
- Rebuilt every frame with **zero allocations**, children created lazily
- Guard on cell size: coincident bodies cannot subdivide forever
- **Explicit stack**, not recursion: bounded depth, no call frames
- Softening stops close encounters from slingshotting particles to infinity when $1/r^2 \rightarrow \infty$

θ is the algorithm's dial

θ	behaviour	cost
$\rightarrow 0$	exact, opens all	$O(N^2)$
0.8	default	$O(N \log N)$
1.6	visible artefacts	fastest

The HUD node count makes the accuracy/speed trade-off legible in real time.



Integration, stability, boundaries

Simulation engine

Semi-implicit (symplectic) Euler — same cost as explicit, far more stable for orbits

- Friction as a **half-life**, not a coefficient: framerate-independent and physically meaningful
- Time-step clamp: returning from a background tab costs slow motion, not an explosion
- Velocity clamp as the last line of defence
- **Two-pass update**: all forces from start-of-step positions, then all positions
a fused loop would make the result depend on particle numbering

Two boundary modes, toggled live

- Reflecting walls with restitution
- Toroidal wrap: positions *and* forces wrap (minimum image + periodic cell sweep)
- Gravity deliberately not wrapped — it would need an Ewald summation



Table of Contents

Creature and visualisation

► Overview

► Simulation engine

► Creature and visualisation

► Interactions

► Wrap-up



The Grazer: hierarchical modelling

Creature and visualisation

root → hip ×4 → thighPivot → knee → shinPivot → foot
five levels deep, plus body, face → eye → pupil/glints, antennae → stalk/bulb

Welded joints by construction

- A cylinder is born centred on the origin, aligned to +y: the wrong frame for a bone
- Geometry baked once at build time — rotated to +x, translated so the **pivot sits at the origin**
- The next joint is simply a child Group at the end of the segment

Segments cannot come apart — not because the animation is careful,
but because the hierarchy makes separation unrepresentable.



Procedural gait, and a creature that is physical

Creature and visualisation

No keyframes anywhere — a closed-form function of a phase accumulator

- Four legs, constant offsets a quarter-cycle apart $\Rightarrow$ a coordinated walk
- Half-wave rectified lift: foot planted during stance, lifted during swing
- Cycle speed tracks body speed
- Secondary motion, all closed-form: hover bob, antennae lean, stochastic blink, damped heading

Not a decorative overlay

- **Solid:** particles are projected out of its surface, impulse only if moving inward
- **Emits a field:** radial attract/repel *plus* a tangential swirl
- The swirl is why particles form an orbiting disc instead of a static ball
- The aura sphere is drawn at exactly the field radius: an honest depiction



Visualising the invisible

Creature and visualisation

- **Octree wireframe** — large cells in empty space, tight subdivision around mass: Barnes–Hut adaptivity becomes obvious in one glance
- **Gravity field streamlines** — up to 8000 samples, faint vector plus a travelling pulse; the per-sample phase comes from position, so the wave sweeps diagonally and never pulses in unison
- **Deforming space lattice** — 9^3 nodes displaced by gravity and by the user's poke, always recomputed from rest positions, so no drift accumulates

The trick that makes it free

Directions rebuilt every fourth frame (15 Hz), pulse redrawn every frame (60 Hz).
The eye cannot tell.



Performance

Creature and visualisation

Stage	Cost
Grid build	$O(N + \text{cells})$, allocation-free
Short-range forces	$O(N)$ expected
Octree build + traversal	$O(N \log N)$, allocation-free
Rendering	one draw call for N particles
Field / lattice	only when enabled, field throttled $4\times$

- Per-instance colour written on type change; only transforms re-uploaded per frame
- Coarse spheres (80 faces): 0.25M triangles at $N = 3000$, the normal map restores the detail
- **Steady-state allocation rate: zero**
- HUD: fps • particles • pairs evaluated • octree nodes • mode



Table of Contents

Interactions

► Overview

► Simulation engine

► Creature and visualisation

► **Interactions**

► Wrap-up



Five registers, starting with the camera

Interactions

Look • Touch • Strike • Inhabit • Legislate

Camera: a hand-written spherical orbit

- ϕ clamped off the poles $\Rightarrow$ `lookAt` never degenerates and flips the view
- Zoom is **multiplicative** $\Rightarrow$ it feels linear at every scale
- Pan speed scales with radius $\Rightarrow$ constant fraction of the screen at any zoom
- Right-drag always orbits $\Rightarrow$ the user is never trapped inside a tool
- Panning silently cancels follow mode $\Rightarrow$ the camera never fights the user



Touch and strike

Interactions

Four drag tools: Orbit • Push • Pull • Warp (swirl + collapse + axial compression)

- Where is the cursor in 3D? Unprojected onto the **camera-facing plane through the orbit target**: the tool acts at the depth you are looking at, from any angle
- Swirl axis captured at drag start $\Rightarrow$ the whirlpool is always face-on
- Quadratic falloff, no hard shell; a marker sphere shows the exact reach

Six one-shot impulses: Shockwave • Implode • Vortex • Warp • Jitter • **Fold**

- **Fold** is a transformation of space, not a force
- It mirrors **position and velocity** $\Rightarrow$ two coherent counter-rotating halves that then collide



Inhabit: piloting the Grazer

Interactions

- WASD + QE + boost; **force-based**, not positional
- Input builds an acceleration in the **creature's local frame**
- Normalised before scaling $\Rightarrow$ diagonals are not $\sqrt{2}$ times faster (the classic bug)
- Same framerate-independent exponential damping as the particle friction
- The result has weight: it accelerates, coasts, and **stirs the soup as it goes**
- Follow camera lerps at 5% per frame — an instant snap would jitter with the hover bob
- The hint bar fades in only while moving: discoverable, not permanent



Legislate: the rule matrix

Interactions

The most consequential interaction is the smallest widget.

- $T \times T$ grid, rows act on columns; colour encodes sign and magnitude
- One click cycles a cell through five discrete values
- Read directly by the force kernel: **no apply button, no reseed**

Two clicks

Set one cell to $+1$ and its mirror to -1 : type 0 chases, type 1 flees, the pair self-propels across the volume. Make the matrix symmetric instead: static clumps.

The user discovers asymmetry in ten seconds, without reading anything.

- **Nine presets** — the preset fixes the *regime*, the rerolled matrix keeps the outcome new
- **Live reconfiguration** — particle count reallocates every buffer; world size *scales* positions instead of clipping them, so structures survive the resize



Table of Contents

Wrap-up

► Overview

► Simulation engine

► Creature and visualisation

► Interactions

► **Wrap-up**



Limitations — all deliberate

Wrap-up

- **Single-threaded, CPU-bound** beyond about 3000 particles
a Web Worker with shared memory needs COOP/COEP headers $\Rightarrow$ needs a server $\Rightarrow$ breaks the `file://` promise
- **Gravity is not periodic** under wrap — an Ewald summation is out of scope
- **Sphere collision proxy**, not the visual silhouette
- **No foot-locking** — a slight skate at high speed; would need inverse kinematics
- **Momentum is not conserved** — by design, and so there is nothing to validate against

Next: GPU-side integration • IK with foot locking • flocking creatures • matrices shareable as URLs



Takeaway

Wrap-up

- Two simple laws at two scales produce cells, chains, filaments, orbits, spirals and swarms — **none of which exists anywhere in the source**
- What makes it interactive rather than merely correct: **each law paired with the structure that fits its scale**
- Pooled typed arrays everywhere $\Rightarrow$ zero steady-state allocation, no GC stutter
- The approximation is not hidden behind the curtain — **the θ slider puts it on stage**
- Fully asset-free: 1 HTML + 1 CSS + 10 JS files, one dependency

Demo



PRIMORDIA *Thank you for listening!*

Any questions?